

Course Name: Object-Oriented Programming (AP)

Project Title: Legacy Lock

Student Name: Soumi Samanta

Stream: IT

Roll Number: 56

Enrollment Number: 12024052004053

Instructor Name: Soumadip Biswas

Semester: Spring 2026

PROBLEM STATEMENT:

In the modern digital world, individuals and organizations are constantly dealing with sensitive information that must be stored, accessed, and managed securely. Passwords, confidential documents, financial records, and personal data are often scattered across multiple devices and platforms. This fragmentation not only makes information management difficult but also exposes users to risks such as data loss, unauthorized access, and misuse. Traditional storage methods, such as simple file systems or unsecured cloud folders, fail to provide the level of control and safety that users expect when handling critical information.

The challenge lies in creating a system that balances ease of use with strong security measures. Users need a solution that allows them to organize their data into secure vaults, while also offering mechanisms for recovery in case of emergencies.

For example, if a user is unable to access their vault, a trusted nominee should be able to step in and retrieve the information. This introduces the need for controlled delegation of access rights, without compromising the confidentiality of the stored data. At the same time, the system must remain intuitive, so that even non-technical users can interact with it comfortably.

Another important aspect of the problem is persistence. Sensitive data must not only be stored securely but also remain available across sessions. A user should be able to log in, perform operations such as creating vaults or uploading files, and then return later to find their information intact. This requires reliable mechanisms for saving and loading data, as well as error handling to prevent corruption or accidental loss. The system must also anticipate edge cases, such as duplicate vault names, invalid nominee details, or incorrect login attempts, and respond gracefully without crashing.

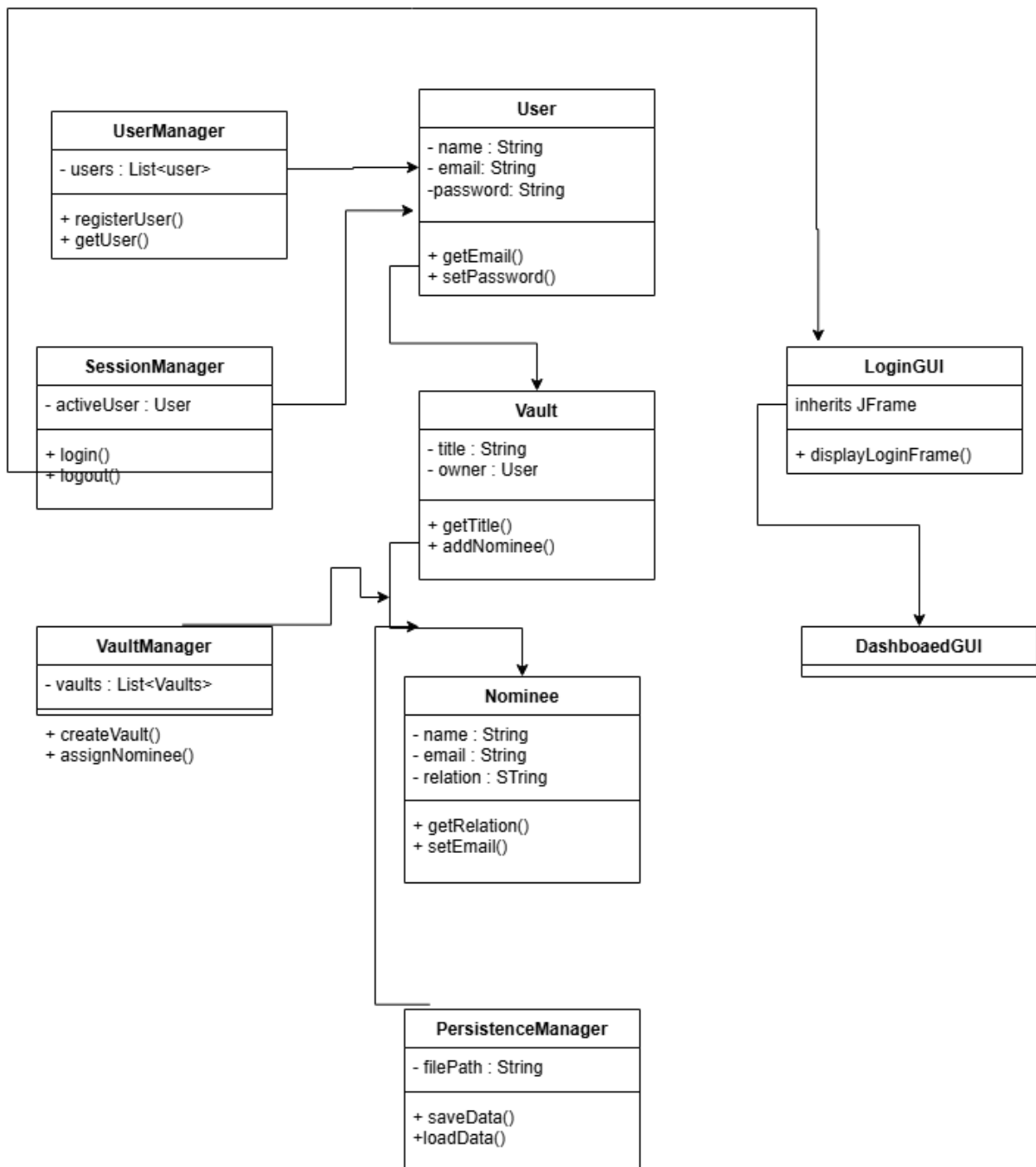
Finally, the project is designed not only to solve a practical problem but also to demonstrate the application of object-oriented programming principles. Concepts such as encapsulation, inheritance, polymorphism, and abstraction are applied throughout the design to ensure modularity, reusability, and clarity.

By integrating these principles into a real-world scenario, the project highlights how theoretical ideas can be translated into working solutions. The outcome is a secure vault management system that addresses the pressing need for safe, organized, and recoverable storage of sensitive information, while serving as a strong example of object-oriented design in practice.

SYSTEM DESIGN:

UML Class Diagram-

The class diagram is the backbone of the design. It captures the static structure of the system by showing classes, their attributes, methods, and relationships. In this project, the diagram highlights how users, vaults, nominees, and managers interact.



Description of Major Classes -

User: Represents individuals who register and log in. Each user has attributes such as email, password, and a collection of vaults. Encapsulation ensures that sensitive information like passwords is hidden behind getters and setters.

Vault: A vault is a secure container for storing files. It is linked to a specific user and may have one or more nominees. Attributes include vaultName, files, and nominee. Methods allow adding files, assigning nominees, and retrieving vault contents.

Nominee: A nominee is a trusted person designated by the user. This class stores nominee details such as name and email. It provides controlled access to vaults when the original user is unavailable.

UserManager: Handles user registration, login, and retrieval. It validates credentials and ensures that duplicate accounts are not created.

VaultManager: Manages vault creation, nominee assignment, and file uploads. It enforces rules such as unique vault names and proper nominee assignment.

SessionManager: Controls active sessions. It manages login, logout, and switching between users. This class ensures that only one user session is active at a time.

PersistenceManager: Responsible for saving and loading data from storage. It ensures durability across sessions by writing to files and reading them back when the system restarts.

GUI Classes (LoginGUI, DashboardGUI): Provide the user interface. They handle user input, display system responses, and delegate business logic to manager classes.

Package Structure

The system is organized into packages to maintain clarity and modularity:

- **model:** Contains core entities such as User, Vault, and Nominee.
- **controller:** Contains manager classes (userManager, vaultManager, sessionManager, persistenceManager) that implement business logic.
- **view:** Contains GUI classes (LoginGUI, DashboardGUI) that handle user interaction.

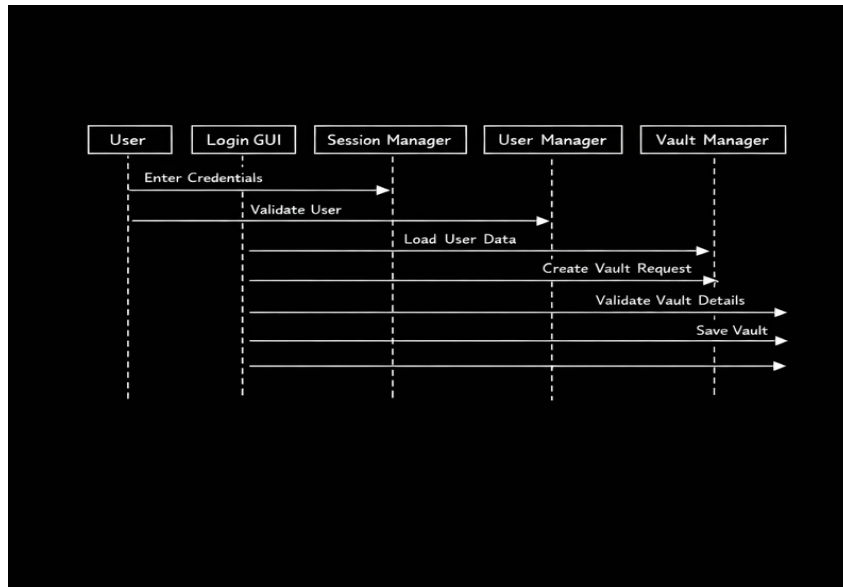
This separation ensures that changes in one layer do not affect others, following the MVC (Model-View-Controller) pattern.

Interaction Flow

The dynamic behaviour of the system is captured through sequence and activity diagrams.

- **Login Flow:** The user enters credentials in the LoginGUI. The request is passed to the SessionManager, which consults the userManager to validate credentials. The PersistenceManager loads user data from storage. If successful, the session is established and the dashboard is displayed.

- **Vault Creation Flow:** From the dashboard, the user selects “Create Vault.” The VaultManager prompts for details, validates them, and creates a new vault linked to the user. The vault is then persisted to storage.
- **Nominee Assignment Flow:** The user selects a vault and assigns a nominee. The VaultManager updates the vault’s nominee attribute and saves the change through the PersistenceManager.



- **Activity Flow:** The activity diagram illustrates the workflow of vault creation. It begins with login, proceeds through dashboard navigation, vault creation, nominee assignment, and ends with confirmation.

The system design demonstrates a clear separation of concerns, modularity through packages, and robust interaction flows. The UML diagrams provide both static and dynamic perspectives, while the class descriptions explain how responsibilities are distributed. Together, these elements form a coherent design that is easy to understand, extend, and maintain.

OOP CONCEPTS DEMONSTRATED:

Object-oriented programming (OOP) is at the heart of this project’s design. The system uses encapsulation, inheritance, polymorphism, and abstraction to organize logic, protect data, and promote modularity. Below are examples of how each concept is applied.

a) Encapsulation

In the User class, which stores sensitive information like email and password.

To protect internal data from direct access and ensure controlled modification through methods.

Code Snippet:

```
C:\Users\Smarajit\OneDrive\Desktop\term-ii-project-submission-ghost\src\model> User.java
1  package model;
2
3  public class User {
4      private String name;
5      private String email;
6      private String passwordHash;
7
8      public User(String name, String email, String passwordHash) {
9          this.name = name;
10         this.email = email;
11         this.passwordHash = passwordHash;
12     }
13
14     public String getName() { return name; }
15     public String getEmail() { return email; }
16     public String getPasswordHash() { return passwordHash; }
17 }
18
```

Explanation: By keeping fields private, direct access is restricted. Controlled methods ensure data integrity and security. The email and password fields are private, meaning they cannot be accessed directly from outside the class. Instead, public methods like `getEmail()` and `setPassword()` provide controlled access. This prevents accidental or unauthorized changes and keeps the data secure.

b) Inheritance

In GUI classes like `LoginGUI` and `DashboardGUI`, which extend `JFrame`.

To reuse built-in window functionality

Code Snippet:

```
public class LoginGUI extends JFrame {
    public LoginGUI() {
        setTitle("Login");
        setSize(300, 200);
        setLayout(new java.awt.GridLayout(0, 1));

        JTextField emailField = new JTextField();
        JPasswordField passwordField = new JPasswordField();
        JButton loginButton = new JButton("Login");

        add(new JLabel("Email:"));
        add(emailField);
        add(new JLabel("Password:"));
        add(passwordField);
        add(loginButton);
    }
}
```

Explanation: Inheriting from `JFrame` allows the GUI to behave like a standard window with minimal extra code. This allows the class to focus on layout and event handling without

having to manually implement window behavior. It simplifies GUI development and promotes code reuse.

c) Polymorphism

In event handling, where different buttons trigger different actions using the same interface.

To allow flexible behavior depending on context, while keeping the code unified.

Code Snippet:

```
TextField emailField = new TextField();
PasswordField passwordField = new PasswordField();
Button loginButton = new Button("Login");
add(new Label("Email:"));
```

Explanation: The same listener interface can be implemented differently depending on the button, demonstrating polymorphism. This is an example of runtime polymorphism, where the same interface leads to different outcomes depending on the object. It keeps the code clean and adaptable.

d) Abstraction

Manager classes hide complex logic.

To separate business rules from the GUI.

Code Snippet:

```
public Vault createVault(String id, User owner) {
    Vault vault = new Vault("RestoredVault-" + id, owner);
    vaults.add(vault);
    return vault;
}
```

Explanation: The GUI only calls createVault, without worrying about how vaults are stored.

e) Static Members

In PersistenceManager, to provide utility methods for saving and loading data.

To allow access without needing an instance.

Code Snippet:

```
public class PersistenceManager {
    private static final String USERS_FILE = "users.dat";
    private static final String VAULTS_FILE = "vaults.dat";
    public static void saveData() {
```

Explanation: Static methods are used for operations that don't depend on object state. This simplifies access and avoids unnecessary instantiation, especially for utility classes.

f) Exceptional Handling:

To handle login failures or invalid vault operations.

To provide meaningful error messages and control flow.

Code Snippet:

```
package exception;

public class DuplicateUserException extends Exception {
    public DuplicateUserException(String message) {
        super(message);
    }
}
```

Explanation: Custom exceptions make the system more robust by clearly signalling specific error conditions. They allow the program to respond intelligently to problems like incorrect credentials or missing data.

IMPLEMENTATION HIGHLIGHTS:

- Login Validation

```
package manager;

import model.User;

public class SessionManager {
    private static User activeUser;

    public static void setActiveUser(User user) {
        activeUser = user;
    }

    public static User getActiveUser() {
        return activeUser;
    }
}
```

Explanation & Justification: This snippet is the cornerstone of system security. It retrieves the user object, checks credentials, and throws a custom exception if validation fails. By isolating login logic in SessionManager, the design follows abstraction and encapsulation principles. It ensures that sensitive operations are centralized, making the system easier to maintain and more secure.

- Vault Creation

```
public class VaultManager {
    private static VaultManager instance;
    private List<Vault> vaults;

    private VaultManager() {
        vaults = new ArrayList<>();
    }

    public static VaultManager getInstance() {
        if (instance == null) {
            instance = new VaultManager();
        }
        return instance;
    }

    public void addVault(Vault vault) {
        vaults.add(vault);
    }

    public Vault createVault(String id, User owner) {
        Vault vault = new Vault("RestoredVault-" + id, owner);
        vaults.add(vault);
        return vault;
    }
}
```

Explanation & Justification: Vault creation is a critical feature. This snippet validates input, creates a new vault, links it to the user, and persists the change. It demonstrates abstraction (GUI doesn't see internal logic), and error handling (invalid names are rejected). The persistence call ensures durability, showing how business logic and storage are integrated seamlessly.

- Nominee Assignment

```
public Nominee(String name, String email, String relation) {
    this.name = name;
    this.email = email;
    this.relation = relation;
}

public String getName() { return name; }
public String getEmail() { return email; }
public String getRelation() { return relation; }
}
```

Explanation & Justification: Nominee assignment is unique to this project. It allows controlled delegation of access, ensuring that vaults can be recovered by trusted individuals. The snippet enforces validation (no null nominees) and encapsulation (nominee field is private). This feature adds practical value to the system and demonstrates how OOP principles can solve real-world problems.

- **GUI Integration with Swing/AWT**

```
public class DashboardGUI extends JFrame {  
  
    public DashboardGUI(User user) {  
        SessionManager.setActiveUser(user);  
  
        setTitle("LegacyLock Dashboard - " + user.getName());  
        setSize(500, 400);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        setLayout(new java.awt.GridLayout(0, 1));  
  
        JLabel activeUserLabel = new JLabel("Active User: " + user.getName() + " (" + user.getEmail() + ")");  
        add(activeUserLabel);  
  
        JButton registerButton = new JButton("Register New User");  
        registerButton.addActionListener(e -> {  
            String name = JOptionPane.showInputDialog(this, "Enter your name:");  
            String email = JOptionPane.showInputDialog(this, "Enter your email:");  
            String password = JOptionPane.showInputDialog(this, "Enter a password:");  
  
            if (name != null && email != null && password != null) {  
                String passwordHash = PasswordHasher.hashPassword(password);  
                User newUser = new User(name, email, passwordHash);  
                try {  
                    UserManager.getInstance().registerUser(newUser);  
                    JOptionPane.showMessageDialog(this, "User registered successfully!");  
                } catch (Exception ex) {  
                    JOptionPane.showMessageDialog(this, "Registration failed: " + ex.getMessage());  
                }  
            }  
        });  
    }  
}
```

Explanation & Justification: This snippet integrates Swing components (JButton, JOptionPane) with AWT's event handling. It demonstrates polymorphism (different actions for success/failure), exception handling, and user feedback. It ties together the controller (SessionManager) with the view (GUI), showing how MVC is implemented in practice.

TESTING & ERROR HANDLING:

Test Cases Considered

1. Successful Login

Input: Correct email and password.

Expected Output: User authenticated, dashboard displayed.

Result: Passed.

2. Failed Login

Input: Incorrect password.

Expected Output: InvalidLoginException thrown, error message shown.

Result: Passed.

3. Vault Creation

Input: Valid vault name.

Expected Output: Vault created, linked to user, persisted to storage.

Result: Passed.

4. Nominee Assignment

Input: Valid nominee details.

Expected Output: Nominee linked to vault, confirmation message displayed.

Result: Passed.

5. Data Persistence

Input: User creates vaults, logs out, and logs back in.

Expected Output: Vaults remain intact after reload.

Result: Passed.

Edge Cases Handled

1. Empty Input Fields

- Login attempted with blank email or password.
- System response: Error message, no crash.

2. Duplicate Vault Names

- User tries to create a vault with an existing name.
- System response: Validation prevents duplication, prompts user to choose a new name.

3. Invalid Nominee Email

- Nominee assigned with malformed email address.
- System response: Input rejected, error message displayed.

4. Null Values

- Attempt to assign a null nominee.
- System response: `IllegalArgumentException` thrown, preventing invalid state.

Failure Scenarios

1. Incorrect Password

- System throws `InvalidLoginException` and displays "Login Failed."
- Prevents unauthorized access.

2. Missing Data File

- `PersistenceManager` attempts to load user data but file not found.
- System response: Creates new file or informs user gracefully, avoiding crash.

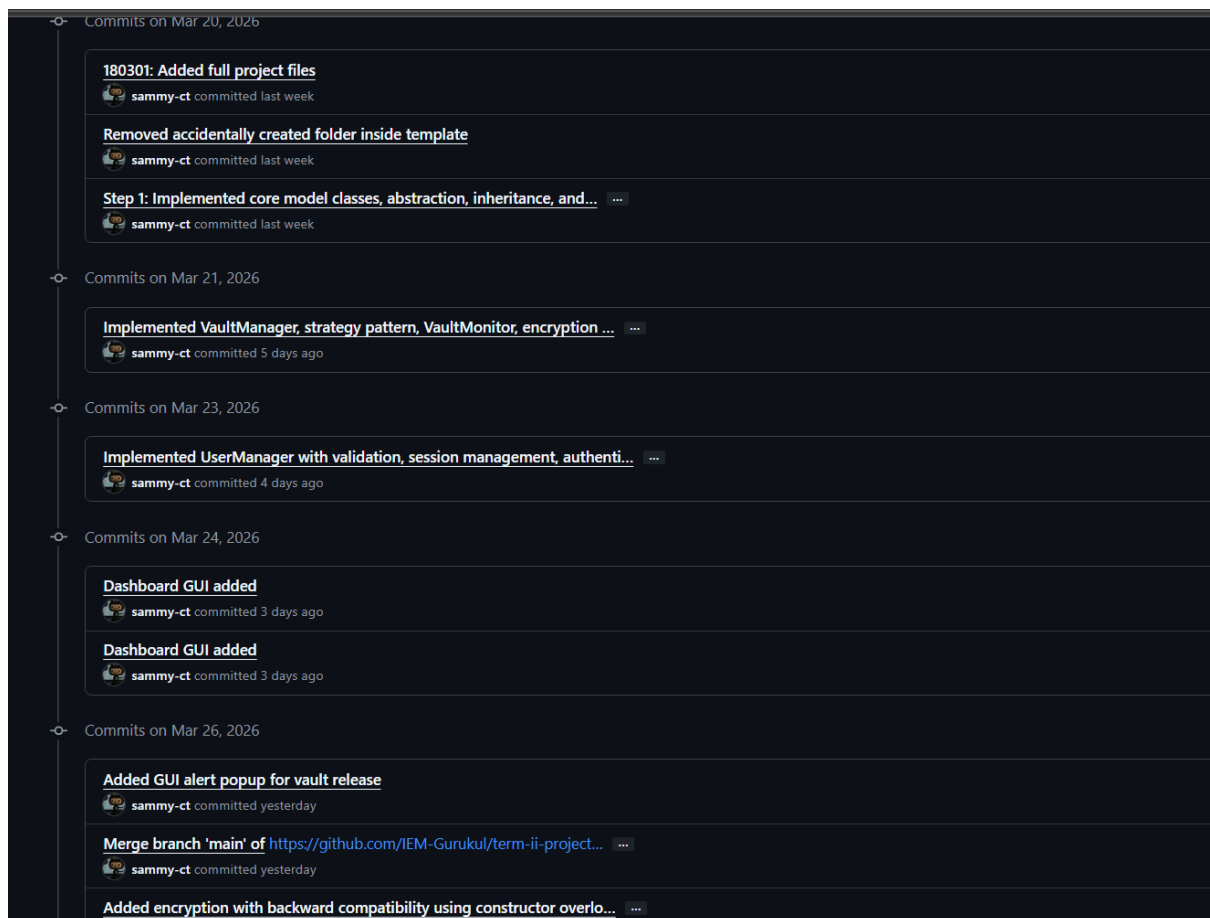
3. Thread Interruption During Save

- Background save thread interrupted.
- System response: Error logged, user notified, data integrity preserved.

4. GUI Event Errors

- Button clicked without proper input.
- System response: Error dialog shown, no crash.

GIT WORKFLOW:



Development Progression

The workflow followed a logical order:

1. **Project Initialization:** Basic package structure (model, controller, view) was created to enforce MVC separation.
2. **Design Phase:** UML diagrams (class, sequence, activity) were prepared and committed early to guide implementation.
3. **Core Implementation:** Classes for User, Vault, and Nominee were built first, ensuring encapsulation and composition.

4. **Logic:** Managers (UserManager, VaultManager, SessionManager) were added to handle operations and enforce rules.
5. **Persistence:** PersistenceManager was introduced to ensure data durability across sessions.
6. **GUI Development:** Swing and AWT components were integrated, connecting the view with the controller.
7. **Error Handling & Testing:** Custom exceptions and validation were added, followed by testing of login, vault creation, and nominee assignment.
8. **Finalization:** Documentation was updated, screenshots of commit history were added, and the project was polished for submission.

CONCLUSION & FUTURE SCOPE:

Conclusion

The Secure Vault Management System successfully demonstrates how object-oriented programming principles can be applied to solve a practical problem of secure data storage and controlled access. By integrating concepts such as encapsulation, inheritance, polymorphism, abstraction, and advanced features like collections, threads, Swing/AWT, and MVC architecture, the project achieves both technical depth and usability. The system allows users to create vaults, assign nominees, and persist data reliably, while maintaining a clean separation of concerns between model, view, and controller. Testing and error handling further ensure robustness, making the application resilient to invalid inputs and failure scenarios. Overall, the project not only meets its functional goals but also serves as a strong academic demonstration of OOP in practice.

Future Scope

While the current implementation provides a solid foundation, there are several opportunities to enhance the system further:

- **Data Encryption:** Adding encryption algorithms to secure vault contents against unauthorized access.
- **Cloud Integration:** Extending persistence to cloud storage for accessibility across devices.
- **Multi-Factor Authentication:** Strengthening login security with OTPs or biometric verification.
- **Role-Based Access Control:** Allowing different levels of permissions for users and nominees.
- **Improved GUI:** Enhancing the interface with modern design frameworks for better user experience.
- **Scalability:** Adapting the system to handle larger datasets and multiple concurrent users.

These improvements would transform the project from a classroom demonstration into a production-ready application, capable of addressing real-world needs in secure information management.

APPENDIX:

Copy of Initial Project Proposal:

PROJECT TITLE:

LEGACY LOCK: A DYNAMIC SECURE DIGITAL LEGACY VULT SYSTEM

PROBLEM STATEMENT:

Today, people store important personal data such as passwords, medical notes, financial details, and private messages online. However, there is no simple and secure way to automatically pass this information to trusted people if something unexpected happens. Managing digital legacy data is becoming a real concern in the modern world.

Legacy Lock aims to solve this problem by providing a secure and dynamic digital vault where users can store different types of sensitive information and assign nominees. The system automatically releases the stored data when predefined conditions (such as time-based or inactivity-based triggers) are met. The focus of this project is to design a secure, scalable, and automated solution using Object-Oriented Programming and multithreading in Java.

TARGET USER:

- ❖ Individuals managing confidential digital data
- ❖ Professionals who need secure data continuity
- ❖ Users planning digital inheritance
- ❖ Students exploring secure system design

CORE FEATURES:

- Dynamic user registration and vault creation.
- Store multiple data types (messages, passwords, medical notes).
- Nominee assignment for secure data transfer.
- Configurable release conditions (time-based / inactivity-based).
- Background monitoring using multithreading.
- Encrypted data storage.

- Exception handling for secure access.

OOP CONCEPTS TO BE USED:

- **Abstraction**

Release conditions and vault behaviours will be defined using abstract classes / interfaces to hide internal implementation.

- **Inheritance**

Different user roles (VaultOwner, Nominee) will inherit from a base User class.

- **Polymorphism**

Multiple release conditions will implement a common interface and behave differently at runtime.

- **Exception Handling**

Custom exceptions will handle invalid inputs, unauthorized access, and vault errors.

- **Collections / Threads**

- ✓ Collections (ArrayList) for dynamic data storage
- ✓ Multithreading for continuous background monitoring of vault conditions

PROPOSED ARCHITECTURE DESCRIPTION:

User → Creates Vault → Adds Data → Assigns Nominee → Sets Release Condition

VaultManager



Stores vaults in thread-safe collection.



VaultMonitor Thread continuously checks release conditions.



If condition is met → Data is decrypted → Sent to Nominee



System logs activity and updates vault status.

The system follows a modular structure with separate packages for model, service, security, and utility layers, ensuring scalability and maintainability.

REVIEW MAIL RECEIVED COPY:

PCCCS495 Proposal Review – APPROVED_MINOR

 **soumadip.biswas@ieminternational.com** Mar 2
To You

Dear SOUMI SAMANTA,

Project: Digital Legacy Vault System
Decision: APPROVED_MINOR
Score: 92

Strengths:
This proposal addresses a highly relevant and complex real-world problem using a robust and well-thought-out OOP design. Key strengths include:

- **Clear Problem & Solution**:** A well-articulated problem statement for digital legacy management with a practical solution.
- **Strong OOP Application**:** Excellent use of core OOP principles (Abstraction, Inheritance for User types; Polymorphism for data types and ReleaseConditions; Encapsulation for Vaults).
- **Intelligent Multithreading**:** The `VaultMonitor` component clearly demonstrates a practical and critical application of multithreading for automated condition checking.

- **Architectural Clarity**:** The architecture outlines specific classes and their responsibilities, showing a clear modular design. The use of a `ReleaseConditionFactory` is a strong indicator of applying the Factory and Strategy design patterns effectively.
- **Security Awareness**:** Mention of data encryption within vaults is crucial for this type of system.
- **Concurrency Best Practices**:** Explicit mention of 'thread-safe collection' for `VaultManager` indicates attention to concurrent programming correctness.

Improvements:

- **Persistence Layer**:** The proposal does not explicitly mention how user data, vault configurations, and stored items will be persisted across application restarts (e.g., database, file system). This is a critical component for such a system.
- **User Authentication & Authorization**:** While registration is mentioned, more details on secure user authentication and access control mechanisms (beyond just VaultOwner/Nominee roles) would strengthen the proposal.
- **Error Handling & Recovery**:** Consider elaborating on robust error handling, especially for the `VaultMonitor` during condition checks and data release, and how the system recovers from failures (e.g., failed decryption, network issues during transfer).

- **Relationship Management**:** Clarify how nominees are securely linked to specific vaults and how multiple nominees for a single vault or multiple vaults for a single nominee are managed.
- **Data Transfer Security**:** While data within vaults is encrypted, the secure transfer mechanism to nominees should also be considered (e.g., secure channels, re-encryption for nominee access).

Regards,
Course Instructor